■ ALASSIO DATA CATALOG

Architecture and Features

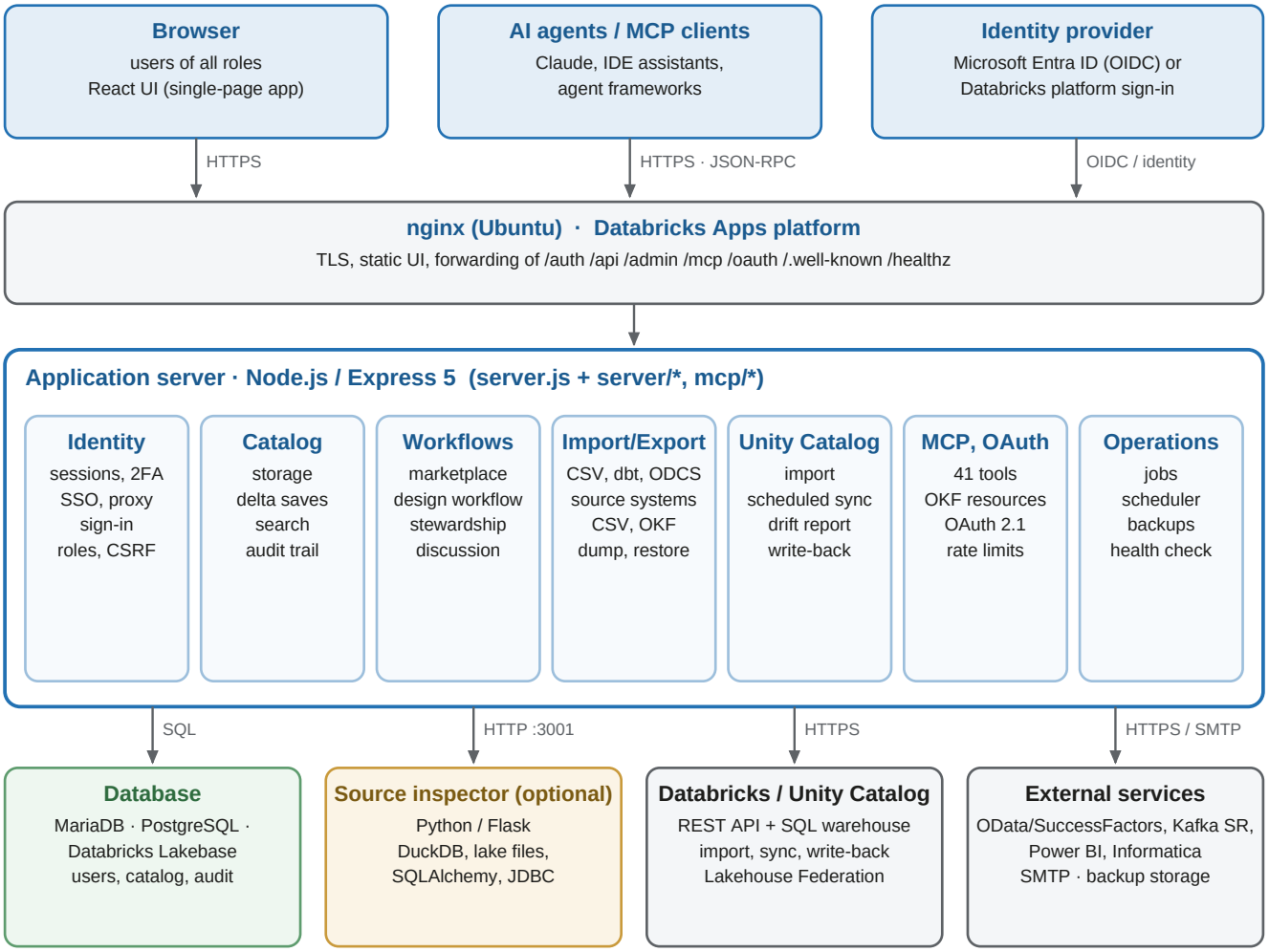
Overview of the Alassio Data Catalog · Version 1.79.6

The Alassio Data Catalog is a self-hosted enterprise data catalog. It describes which data exists, where it lives, who is responsible for it and how it connects – technically (databases, tables, columns, lineage) and in business terms (definitions, glossary, data products). In the marketplace people request access and design new data products in a multi-stage workflow; AI agents read and maintain the catalog through the Model Context Protocol.

This document is for architects, operators and business owners and describes the structure, data model, security and features.

Technology	React 19 / Vite (UI) · Node.js / Express 5 (server) · MariaDB or PostgreSQL / Lakebase · optional Python (inspector)
Operation	own Ubuntu server behind nginx or Databricks App – the same code, differing only in configuration
Languages	UI, help and notifications in English, German, Polish, Spanish, French and Italian
Interfaces	REST API of the UI · MCP server (41 tools, OKF resources) · Unity Catalog · CSV, OKF, dbt, ODCS · source system connectors

1 · System architecture



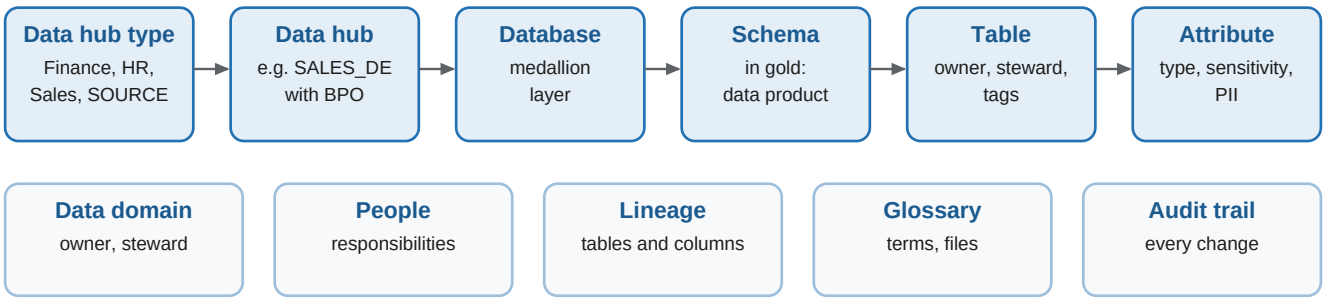
The UI is a single-page application that talks only to its own server – Databricks, source systems and mail servers are reached exclusively from the server. The server is a single Node.js process organised internally into domain modules; routes only handle input and output, the rules live in the modules. An ESLint rule enforces this layering (routes never import routes, domain modules never import routes, the UI never imports server code).

Components in detail

Component	Structure	Responsibilities
UI	React 19, Vite; one view per page, hooks for flows, pure logic in `src/lib`; contexts for catalog data, navigation and actions	navigation of the catalog tree, faceted search, editing, dashboards, marketplace, administration, help and printable handbook; accessible (focus handling, keyboard) and usable on mobile
Application server	Express 5; `server.js` as composition root, domain modules in `server/`, routes in `server/routes/`	sign-in, permission checks, saving the catalog as deltas, workflows, imports and exports as background jobs, Unity Catalog integration, scheduler
MCP server	`mcp.js` (protocol) and `mcp/tools/*`; Streamable HTTP, JSON-RPC 2.0	41 tools for reading, search, lineage, audit, maintenance, glossary, stewardship and discussion; the catalog as an OKF bundle (resources `okf://`)
Database layer	`server/db.js`: one SQL dialect in the code, translated for PostgreSQL (DDL, full-text search via tsvector, upserts)	MariaDB (default), PostgreSQL or Databricks Lakebase with token sign-in through the service principal

Component	Structure	Responsibilities
Native connectors	`server/connectors/` (Node)	OData v2/v4 incl. SAP SuccessFactors, Kafka Schema Registry (Avro, JSON Schema, Protobuf), Power BI Scanner API, Informatica Catalog API
Source inspector	Python / Flask, mode registry in `inspector/connectors/`	DuckDB files, data lake files (Parquet, CSV, JSON, Delta, Iceberg; local, S3, Azure), SQL databases via SQLAlchemy, generic JDBC

2 · Data model



The catalog is hierarchical: **data hub type** → **data hub** → **database** → **schema** → **table** → **attribute**. Data hub types group by business segment; the SOURCE type for generic source systems always exists and is listed last in the tree. Databases carry a medallion layer (bronze, silver, gold) that their tables inherit; schemas in gold databases are **data products**. Responsibility is inherited downwards (BPO of the data hub → domain → database → schema → table) and can be overridden on every level.

Area	Tables
Catalog	`cat_hubs`, `cat_databases`, `cat_schemas`, `cat_tables`, `cat_attributes`, `cat_domains`, `cat_people`, `cat_lineage`, `cat_glossary`, `catalog_meta`
Users and access	`users`, `sessions`, `invites`, `password_resets`, `agent_keys`, `oauth_clients`, `oauth_codes`, `oauth_tokens`
Collaboration	`marketplace_requests`, `request_files`, `messages`, `comments`, `ratings`, `usage_events`, `glossary_files`
Operations	`audit`, `jobs`, `backups`, `settings`

Every change is recorded field by field with user, timestamp and old and new value in the audit trail. This drives the version history with restore and the attribution of changes to people for the write-back to Unity Catalog. Above 5,000 attributes the UI loads columns only when a table is opened; exports and KPIs are computed in full on the server.

3 · Roles and permissions

Role	May
Admin	everything, including user management, SSO, backups, synchronisation and reading source systems
Data Steward	create, change and delete all catalog objects, maintain the glossary, imports, stewardship
Project Manager	design data products and drive them through the workflow, request business descriptions
Business Analyst / Req. Engineer	refine designs, document the sourcing (phases 1 and 2)

Role	May
Data Ingestion	confirm the ingested tables (phase 1)
Development & Test / Sign-off	develop, test, sign off (phase 3)
Guest	read, request access, discuss, rate
Agent	technical account for MCP with read or write access, without a governance role

A person can hold several roles; the highest applies. Responsibilities count as well: owners and stewards may maintain, certify and attest the objects in their area. Roles can be derived from identity provider groups (Entra ID, proxy header or Databricks groups).

4 · Features

Catalog and search

- Tree, cards and detail pages for every level; technical and business metadata on separate tabs.
- Relevance-ranked full-text search across names, descriptions, definitions, terms, tags, data types and stewards, with facets for object type, hub, layer and sensitivity.
- Dashboard with KPIs and governance coverage (owner, steward, descriptions, sensitivity).

Governance and stewardship

- Governance Overview for domains, people and assignments; PII flag per attribute.
- Certify, deprecate (with successor), review cycles (90/180/365 days) and ownership attestation with reminders.
- Project and use case tags on tables and data products.

Lineage and glossary

- Table lineage with column mapping; view of a data product's complete feeding chain.
- Glossary with definitions, linked catalog objects, URLs and file attachments.

Marketplace and workflows

- Access requests for data products, decided by admins, with notification.
- Design of new data products across three phases with return, withdrawal, file attachments and the status overview “Discover Data Products”.
- Personal dashboard with everything awaiting an action; in-app inbox and optional e-mail in the recipient's language.

Collaboration

- Discussion per table, questions to steward and owner with notification, star ratings, usage statistics.

Imports and exports

- CSV import and export (one row per attribute, also for hundreds of thousands of rows as a background job).
- dbt projects (manifest.json, catalog.json) and ODCS data contracts; source systems via the inspector and native connectors.
- Export as Open Knowledge Format (Markdown bundle for people and AI); full dump and restore.

Unity Catalog

- Import of catalogs including lineage and governance tags; scheduled synchronisation with a drift report or automatic apply.
- Write-back of descriptions, tags and properties: every user sees a preview and writes back only their own changes.

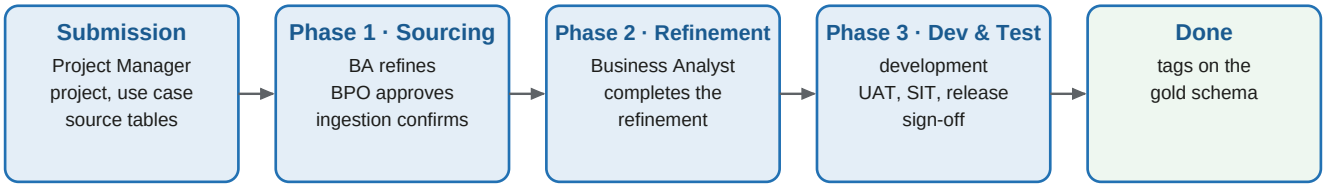
AI agents

- MCP server with the same rules as the UI; authentication by agent key, OAuth 2.1 (as the person) or platform identity; every agent change in the audit trail.

Operations

- Daily backups with restore drill, stored locally, in S3, Azure Blob or a Unity Catalog volume.
- Health check `/healthz`, list of background jobs, tasks via API for external schedulers, graceful shutdown.

5 · Data product workflow



Every step can be returned with a reason; a design can be withdrawn with a reason until “Done”. Everyone involved is notified.

A design passes through the phases in order; every role sees its open steps on the personal dashboard. Selected source tables receive the Data Product, Project and Use Case tags as soon as the design is submitted; on completion the gold schema is tagged. BPMN models of the processes are available under `/docs` of the application.

6 · Security

Area	Implementation
Sign-in	passwords hashed with scrypt; TOTP two-factor authentication; slow-down after failed attempts and lock-out after ten; Entra ID (OIDC) with enforceable SSO and break-glass accounts; proxy sign-in for Databricks Apps
Sessions and tokens	session, invitation and reset tokens stored only as SHA-256 hashes; a password change ends other sessions
Web security	CSRF protection (session-bound token, origin check), Content-Security-Policy, X-Frame-Options, nosniff, HSTS behind HTTPS
Secrets	stored Databricks and Entra secrets encrypted with AES-256-GCM (`SECRETS_KEY`)
Outbound connections	SSRF protection: connectors reach no private addresses, Databricks only allowed workspace hosts
MCP	schema validation of every tool call, rate limits per credential, write access only for admins and stewards, no user management via MCP
Traceability	field-level audit trail for people and agents; excluded from search engines

7 · Deployment options

	Ubuntu server	Databricks App
Web server	nginx with Let's Encrypt	platform
Database	MariaDB or PostgreSQL	Lakebase
Sign-in	password + 2FA, Entra ID	Databricks identity
Source systems	inspector + native connectors	native connectors, Lakehouse Federation
Backups	local, S3 or Azure	Unity Catalog volume
Scheduler	in the server	Databricks job
Guide	Deployment Guide · Ubuntu	Deployment Guide · Databricks App